

Topic-1: Inheritance in JAVA

- Inheritance allows one class to inherit the methods and variables from other classes, thus reusing the codes.
- The class which inherits the properties of other is known as subclass (derived class, child class) and the class whose properties are inherited is known as superclass (base class, parent class).
- To implement inheritance in Java, the “**extends**” keyword is used. It inherits the properties (attributes or/and methods) of the base class to the derived class. The word "extends" means to extend functionalities i.e., the extensibility of the features.
- Inheritance establishes an “is-a” relationship between two classes or a “parent-child” relationship.

Terminology to understand inheritance:

- **Class:** A blueprint or template that defines the properties and behaviours of objects.
- **Sub class/Child class/Derived class:** A class that inherits from another class. It extends the functionality of the superclass by adding new methods or overriding existing ones.
- **Super class/Parent class/Base Class:** The class from which another class inherits. It provides a base for subclasses to build upon.
- **Reusability:** The ability to reuse code from an existing class in a new class, reducing redundancy and promoting efficiency.

Topic-2: Types of Relationships

2.1 Is-A Relationship in Java:

- The is-a relationship, also known as the inheritance relationship, represents a type of relationship between two classes where one class is a specialized version of another.
- It implies that a subclass is a specific type of its superclass.
- For example, a Potato **is a** Vegetable, a Bus **is a** Vehicle, a Bulb **is an** ElectronicDevice, Dog **is an** Animal and so on.
- One of the properties of inheritance is that inheritance is unidirectional in nature. Like we can say that a house is a building. But not all buildings are houses.
- The is-a relationship is implemented in Java using the `extends` keyword.

Example:

```
// Base class
class Fruit {
    private String fruitName;
    public void setFruitName(String fruitName) {
        this.fruitName = fruitName;
    }
    public String getFruitName() {
        return this.fruitName + " is a Fruit";
    }
}
```

```
// Derived class
class Mango extends Fruit {
    public static void main(String gg[]){
        // base class stores the reference of instance of derived classes
        Fruit fruit = new Mango();
        System.out.println("Fruit name is Mango");
        fruit.setFruitName("Mango");
        System.out.println(fruit.geFruitName());
    }
}
```

Output

```
Fruit name is Mango
Mango is a Fruit
```

Benefits of using the is-a relationship in Java:

- **Code reuse:** The is-a relationship allows us to reuse code by inheriting the properties and methods of a superclass. This can save us a lot of time and effort when writing code.
- **Reduced redundancy:** The is-a relationship can help us to reduce redundancy in our code by avoiding the need to write the same code in multiple classes.
- **Increased flexibility:** The is-a relationship can help us to make our code more flexible by allowing us to change the behavior of a subclass without affecting the superclass.
- **Enhanced readability:** The is-a relationship can help to make our code more readable by making it clear how the different classes are related to each other.

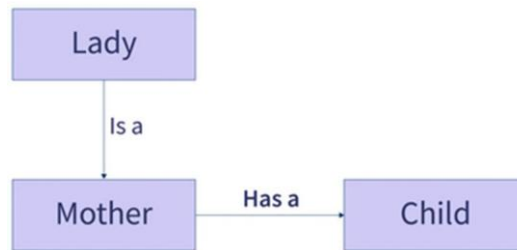
2.2 Has-A Relationship in Java

- Has-A-relationship can be achieved using composition.
- Composition is a term where we declare the object of one class as a data member inside another class.
- In simple words, a relationship where an object has a reference to another instance of the same class or has a reference to an object of another class is called composition or has-a-relation in Java.
- To create has-a-relation we need a minimum of two classes, where the instance variable of one class is declared in another class.

Examples of Has-A-Relationship

Example-1:

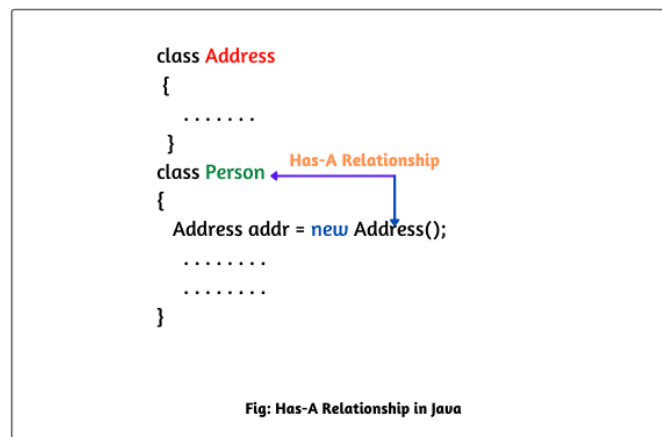
Consider a lady; she can be called a mother only when she has a child. And a child cannot exist without their mother. We can understand the above example by the following diagram.



- In the above diagram, we can see that the mother class is derived from the lady class. So this type of relationship is called an Is-a relationship.
- But our main focus is on the Has-a relationship that exists between mother and child. These two classes are tightly coupled, which means that the existence of one class depends on another class.
- If the child does not exist, then a lady cannot be called a mother and vice versa.

Example-2:

- A most common real-time example of Has-A relationship in Java is “A person has an address”.
- Person class has an instance variable of type Address, as shown in the below code. An instance of Address class is created outside of Person class.



Program:

```
class Address {
    private String street;
    private String city;
    private String state;
    private String zipCode;

    // Define a constructor.
    public Address(String street, String city, String state, String
zipCode) {
        this.street = street;
```

```
        this.city = city;
        this.state = state;
        this.zipCode = zipCode;
    }
    public String getAddressInfo() {
        return street + ", " + city + ", " + state + " " + zipCode;
    }
}
// Define the Person class using composition
class Person {
    private String name;
    private Address address; // Has-A relationship with Address

    public Person(String name, Address address) {
        this.name = name;
        this.address = address;
    }
    public String getPersonInfo() {
        return "Name: " + name + "\nAddress: " +
address.getAddressInfo();
    }
}
}
public class Main {
    public static void main(String[] args)
    {
        // Create an Address instance.
        Address personAddress = new Address("123 Besant Road",
"Vijayawada", "Andhrapradesh", "520002");

        // Create a Person instance with the Address instance.
        Person person = new Person("Meghana", personAddress);

        // Display person's information.
        System.out.println(person.getPersonInfo());
    }
}
```

Output:

Name: Meghana

Address: 123 Besant Road, Vijayawada, Andhrapradesh 520002

Topic-3: Types of Inheritance in Java

The different types of inheritance are observed in Java:

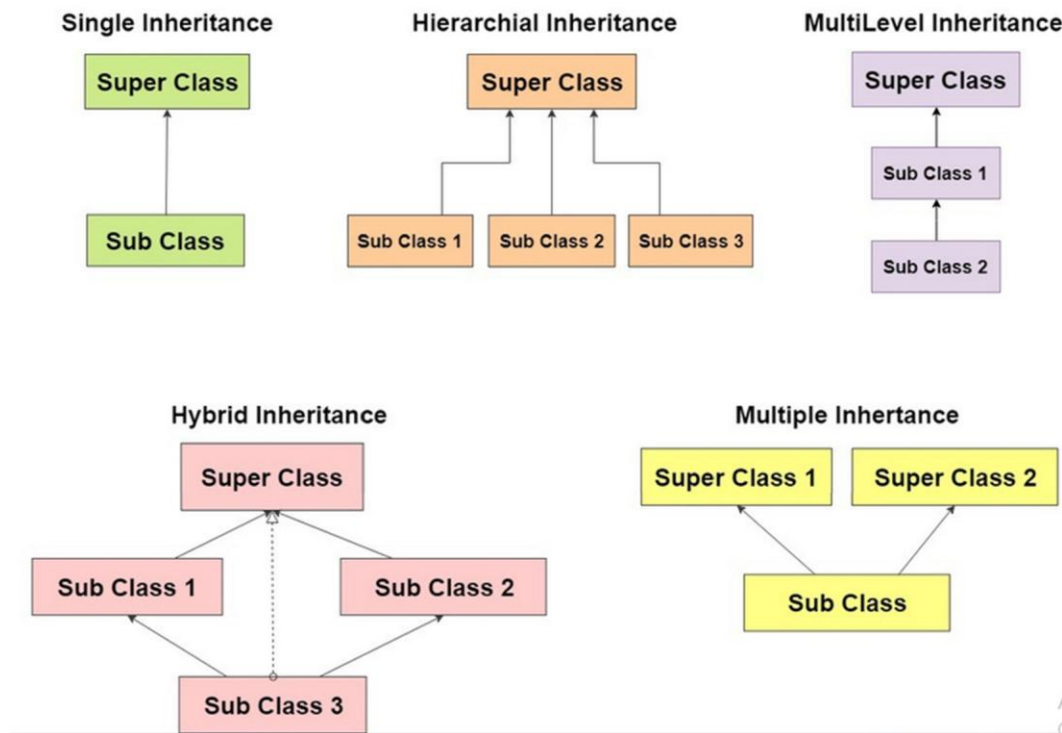


Figure: 1

3.1. Single level inheritance in Java

Single Inheritance



```
class A {
}
class B extends A {
}
```



- As the name suggests, this type of inheritance occurs for only a single class. Only one class is derived from the parent class.
- In this type of inheritance, the properties are derived from a single parent class and not more than that.
- As the properties are derived from only a single base class the reusability of a code is facilitated along with the addition of new features.

Example: An example of a code applying single-level inheritance

```
// Base class (superclass)
class Animal {
    // Method of the base class
    public void eat() {
        System.out.println("This animal eats food.");
    }
}
```

```
// Derived class (subclass)
class Dog extends Animal {
    // Method of the derived class
    public void bark() {
        System.out.println("The dog barks.");
    }
}
```

```
// Driver class to test inheritance
class SingleInheritance {
    public static void main(String[] args) {
        // Create an instance of the derived class
        Dog myDog = new Dog();

        // Call the method from the base class
        myDog.eat();

        // Call the method from the derived class
        myDog.bark();
    }
}
```

Output:

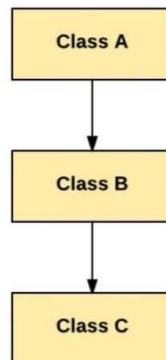
The animal eats food.

The dog barks.

3.2. Multi-level Inheritance in Java

The multi-level inheritance includes the involvement of at least two or more than two classes. One class inherits the features from a parent class and the newly created sub-class becomes the base class for another new class.

As the name suggests, in the multi-level inheritance the involvement of multiple base classes is there. In the multilevel inheritance in java, the inherited features are also from the multiple base classes as the newly derived class from the parent class becomes the base class for another newly derived class.



From the flow diagram , we can observe that Class B is a derived class from Class A, and Class C is further derived from Class B.

Class C is inheriting the methods and properties of both Class A and Class B.

Example: An example of multi-level inheritance

```
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}

class BabyDog extends Dog {
    void weep() {
        System.out.println("Baby Dog is weeping");
    }
}

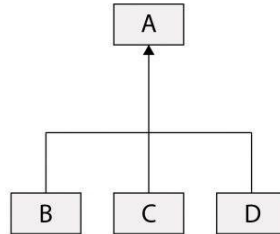
public class MultilevelInheritance {
    public static void main(String[] args) {
        BabyDog bd = new BabyDog();
        bd.weep();
        bd.bark();
        bd.eat();
    }
}
```

Output:

Baby Dog is weeping
Dog is barking
Animal is eating

3.3. Hierarchical Inheritance in Java**Hierarchical Inheritance**

```
class A {  
}  
class B extends A {  
}  
class C extends A {  
}  
class D extends A
```



The type of inheritance where many subclasses inherit from one single class is known as Hierarchical Inheritance.

It is different from the multilevel inheritance, as the multiple classes are being derived from one superclass. These newly derived classes inherit the features, methods, etc, from this one superclass. This process facilitates the reusability of a code and dynamic polymorphism (method overriding).

Example: An example of code showing the concept of hierarchical inheritance

```
class Animal{  
    void eat(){System.out.println("eating...");}  
}  
class Dog extends Animal{  
    void bark(){System.out.println("barking...");}  
}  
class Cat extends Animal{  
    void meow(){System.out.println("meowing...");}  
}  
class TestInheritance3{  
    public static void main(String args[]){  
        Cat c=new Cat();  
        c.meow();  
        c.eat();  
        //c.bark();//C.T.Error  
    }  
}
```

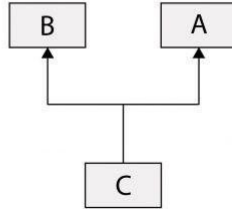

Output:

meowing...

eating...

3.4. Multiple Inheritance in Java**Multiple Inheritance**

```
class A {  
}  
class B {  
}  
class C extends A, B {  
}
```



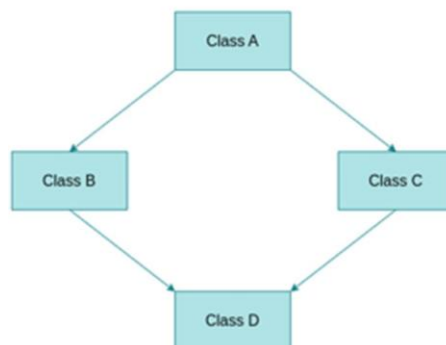
- Multiple inheritances is a type of inheritance where a subclass can inherit features from more than one parent class.
- Multiple inheritances should not be confused with multi-level inheritance, in multiple inheritances the newly derived class can have more than one superclass.
- In java, multiple inheritances can be achieved through interfaces.

Diamond Problem

The **diamond problem** is the classic issue that can arise with multiple inheritance. It occurs when the class inherits from the two classes that have a common ancestor. This situation forms the diamond-shaped inheritance hierarchy. It can lead to ambiguity in the method resolution.

Consider the scenario where class D inherits from classes B and C which both inherit from Class A. This creates the diamond-shaped inheritance hierarchy.

Daimond Problem in Java



Example:

```
class A {
    void check() {
        System.out.println("Method foo() in class A");
    }
}

class B extends A {
    void check() {
        System.out.println("Method foo() in class B");
    }
}

class C extends A {
    void check() {
        System.out.println("Method foo() in class C");
    }
}

class D extends B {
    // Inherits foo() from class B
}
/*
class E extends B,C {    // Error: Multiple inheritance not allowed in
Java                      // Some methods and members
}*/

public class Main {
    public static void main(String[] args) {
        D d = new D();
        d.check();

        // Output: Method check() in class B

        // Uncommenting the below code will result in a compilation
error
        /*
        E e = new E();
        e.check(); // Compilation Error: Reference to 'check()' is
ambiguous
        */
    }
}
```

How Multiple Inheritance is supported in Java?

java can provides the alternative approaches to achieve the code reuse and extensibility without resorting to the multiple inheritance.

1. Interfaces:

In Java, We can use Interfaces to define the contract for the set of behaviors without implementing them. Classes can implement the multiple interfaces and it can allowing them to inherit the method signatures from the multiple sources.

```
// Java Program to Implement
// MultiLevel Inheritance

// Interface 1
interface InterfaceA {
    void methodA();
}

// Interface 2
interface InterfaceB {
    void methodB();
}

// Class
class MyClass implements InterfaceA, InterfaceB {
    public void methodA() {
        // Implementation of methodA
        System.out.println("A");
    }

    public void methodB() {
        // Implementation of methodB
        System.out.println("B");
    }
}

class Main{
    public static void main(String args[]){
        // Inheriting Properties from InterfaceA
        InterfaceA x=new MyClass();

        // Inheriting Properties from InterfaceB
        InterfaceB y=new MyClass();

        x.methodA();
        y.methodB();
    }
}
```

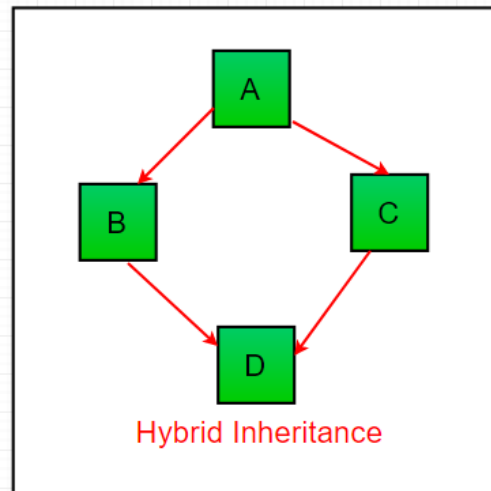
Output

A
B

3.5. Hybrid Inheritance in Java

Hybrid inheritance is a combination of more than two types of inheritances single and multiple. It can be achieved through interfaces only as multiple inheritance is not supported by Java. It is basically the combination of simple, multiple, hierarchical inheritances.

Figure : Representation of a hybrid inheritance



Topic-4: Method Overriding in Java

- If the same method is defined in both the superclass and the subclass, then the method of the subclass class overrides the method of the superclass. This is known as method overriding.

Rules for Method Overriding

- The method in the subclass must have the same name, return type, and parameters as the method in the superclass.
- It allows a subclass to provide a specific implementation for a method that is already defined in its superclass.
- We cannot override the method that is declared as `final`.

Example:

```
class Animal {
    public void displayInfo() {
        System.out.println("I am an animal.");
    }
}

class Dog extends Animal {
    @Override
    public void displayInfo() {
        System.out.println("I am a dog.");
    }
}
```

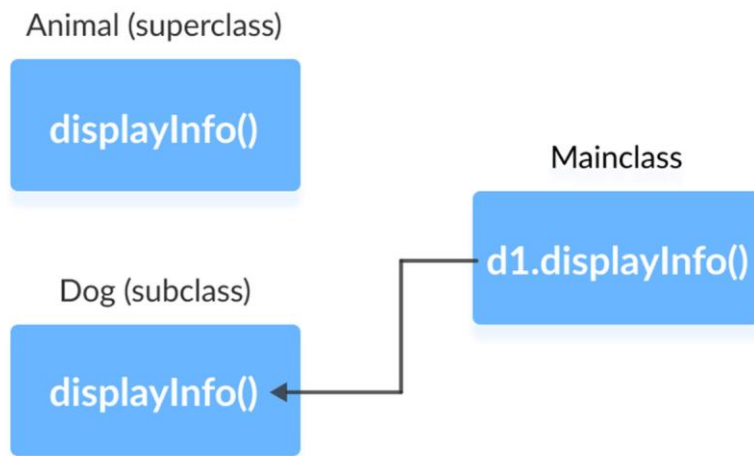
```
class Main {  
    public static void main(String[] args) {  
        Dog d1 = new Dog();  
        d1.displayInfo();  
    }  
}
```

Output:

I am a dog.

In the above program, the `displayInfo()` method is present in both the *Animal* superclass and the *Dog* subclass.

When we call `displayInfo()` using the *d1* object (object of the subclass), the method inside the subclass *Dog* is called. The `displayInfo()` method of the subclass overrides the same method of the superclass.



Notice the use of the `@Override` annotation in our example. In Java, annotations are the metadata that we used to provide information to the compiler. Here, the `@Override` annotation specifies the compiler that the method after this annotation overrides the method of the superclass.

It is not mandatory to use `@Override`.

Topic-5: Upcasting and Downcasting in Java

5.1 Upcasting:

- Upcasting is the process of casting an object of a subclass type to its superclass type. It's done automatically by Java Compiler.
- Upcasting is Implicit and safe.

- It allows you to refer to a subclass object using a superclass reference.

Example:

```
class Vehicle{
    void drive() {
        System.out.println("Driving a vehicle...");
    }
}
class Car extends Vehicle{
    void drive() {
        System.out.println("Driving a car...");
    }
    void speedUp() {
        System.out.println("Speeding up a car...");
    }
}
class UpcastingDemo
{
    public static void main(String[] args) {
        Vehicle v=new Car();
        v.drive();
    }
}
```

Output:

Driving a car...

5.2 Downcasting:

- Downcasting is the reverse process where you cast a superclass reference back to a subclass reference.
- This is not safe and must be done explicitly.
- If the object being cast is not an instance of the subclass, a `ClassCastException` will be thrown at runtime.

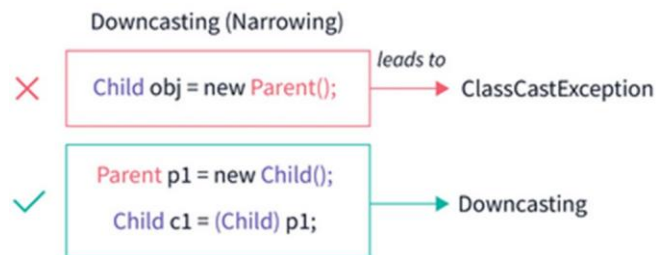
Example:

```
class Vehicle{
    void drive() {
        System.out.println("Driving a vehicle...");
    }
}
class Car extends Vehicle{
    void drive() {
        System.out.println("Driving a car...");
    }
    void speedUp() {
        System.out.println("Speeding up a car...");
    }
}
```

```
class DowncastingDemo
{
    public static void main(String[] args) {
        Car c =(Car) new Vehicle();
        c.drive();
        c.speedUp();
    }
}
```

Output:

Exception in thread "main" java.lang.ClassCastException: class Vehicle cannot be cast to class Car

Solution:**Example:**

```
class Vehicle{
    void drive() {
        System.out.println("Driving a vehicle...");
    }
}
class Car extends Vehicle{
    void drive() {
        System.out.println("Driving a car...");
    }
    void speedUp() {
        System.out.println("Speeding up a car...");
    }
}
class DowncastingDemo
{
    public static void main(String[] args) {
        Vehicle v=new Car();
        Car c=(Car)v;
        c.drive();
        c.speedUp();
    }
}
```

Output:

Driving a car...
Speeding up a car...

Topic-6: 'super' Keyword in Java

The `super` keyword in Java is used in subclasses to access superclass members (attributes, constructors and methods).

Uses of super keyword

1. To call methods of the superclass that is overridden in the subclass.
2. To access attributes (fields) of the superclass if both superclass and subclass have attributes with the same name.
3. To explicitly call superclass no-arg (default) or parameterized constructor from the subclass constructor.

6.1. Access Overridden Methods of the superclass

`super` keyword can be used to access the overridden method of superclass

Example:

```
class Animal {
    // overridden method
    public void display() {
        System.out.println("I am an animal");
    }
}

class Dog extends Animal {
    // overriding method
    @Override
    public void display() {
        System.out.println("I am a dog");
    }

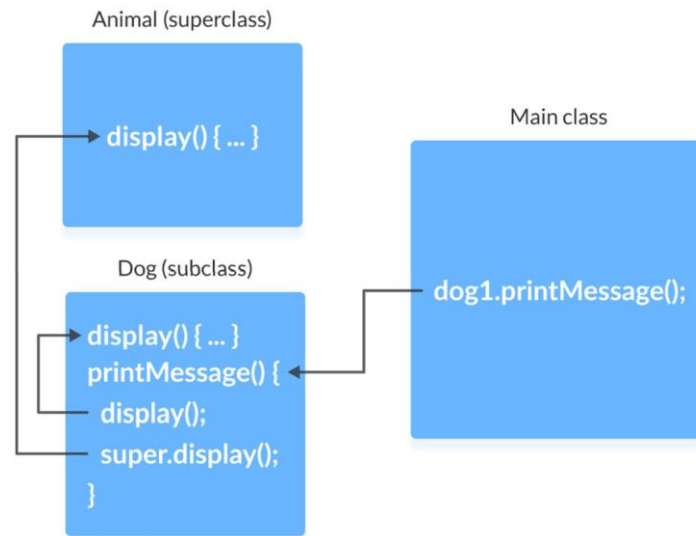
    public void printMessage() {
        // this calls overriding method
        display();

        // this calls overridden method
        super.display();
    }
}

class Main {
    public static void main(String[] args) {
        Dog dog1 = new Dog();
        dog1.printMessage();
    }
}
```

Output

```
I am a dog
I am an animal
```

6.2. Access Attributes of the Superclass

The superclass and subclass can have attributes with the same name. We use the `super` keyword to access the attribute of the superclass.

Example:

```

class Animal {
    protected String type="animal";
}

class Dog extends Animal {
    public String type="mammal";

    public void printType() {
        System.out.println("I am a " + type);
        System.out.println("I am an " + super.type);
    }
}

class Main {
    public static void main(String[] args) {
        Dog dog1 = new Dog();
        dog1.printType();
    }
}
  
```

Output:

```

I am a mammal
I am an animal
  
```

6.3. Access superclass constructor

- As we know, when an object of a class is created, its default constructor is automatically called.
- To explicitly call the superclass constructor from the subclass constructor, we use `super()`. It's a special form of the `super` keyword.
- `super()` can be used only inside the subclass constructor and must be the first statement.

Calling Default (or) No-arg Constructor of superclass

Example:

```
class Animal {  
  
    // default or no-arg constructor of class Animal  
    Animal() {  
        System.out.println("I am an animal");  
    }  
}  
  
class Dog extends Animal {  
  
    // default or no-arg constructor of class Dog  
    Dog() {  
  
        // calling default constructor of the superclass  
        super();  
  
        System.out.println("I am a dog");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Dog dog1 = new Dog();  
    }  
}
```

Output

```
I am an animal  
I am a dog
```

Calling Parameterized Constructor of superclass

```
class Animal {  
  
    // default or no-arg constructor  
    Animal() {  
        System.out.println("I am an animal");  
    }  
}
```

```
// parameterized constructor
Animal(String type) {
    System.out.println("Type: "+type);
}

class Dog extends Animal {

    // default constructor
    Dog() {

        // calling parameterized constructor of the superclass
        super("Animal");

        System.out.println("I am a dog");
    }
}

class Main {
    public static void main(String[] args) {
        Dog dog1 = new Dog();
    }
}
```

Output

```
Type: Animal
I am a dog
```

The compiler can automatically call the no-arg constructor. However, it cannot call parameterized constructors.

If a parameterized constructor has to be called, we need to explicitly call it in subclass constructor.

Topic-7: Behavior of Access Modifiers in Java Inheritance

Superclass members can be inherited to subclass provided they are eligible by *access modifiers*. The behavior of access modifiers in the case of inheritance in Java is as follows:

1. The **private** members of the superclass cannot be inherited to the subclass because the private members of superclass are not available to the subclass directly. They are only available in their own class.
2. The **default** members of the parent class can be inherited from the derived class within the same package.
3. The **protected** members of a parent class can be inherited to a derived class, but the usage of protected members is limited within the package.
4. The **public** members can be inherited from all subclasses.

Example Program-1:

private members of superclass are not accessible or visible in subclass but protected members are available in subclass

```
package inheritance;
public class Baseclass {
    private int x = 30;
    protected int y = 50;
    private void m1() {
        System.out.println("Base class m1 method");
    }
    protected void m2() {
        System.out.println("Base class m2 method");
    }
}
public class Derivedclass extends Baseclass {

}
public class MainClass {
    public static void main(String[] args) {
        // Creating an object of subclass.
        Derivedclass d = new Derivedclass();

        // We cannot access private members due to not being available in
        the subclass.
        // System.out.println("x = " +d.x);
        // d.m1();

        // Accessing the protected members from the subclass.
        d.m2();
        System.out.println("y = " +d.y);
    }
}
```

Output:

```
Base class m2 method
y = 50
```

Example Program-2:

default and public members of superclass that are accessible in the subclass.

```
package inheritance;
public class Identity {
    String name = "Deep"; // default member.
    public int age = 28; // public member.

    // Default method.
    void m1() {
        System.out.println("Name: " +name);
    }
    // Public method.
    public void m2() {
        System.out.println("Age: " +age);
    }
}
```

```
public class Person extends Identity {  
  
}  
public class Mytest {  
public static void main(String[] args){  
    Person p = new Person();  
    p.m1();  
    p.m2();  
    System.out.println("Name: " +p.name);  
    System.out.println("Age: " +p.age);  
}  
}
```

Output:

```
Name: Deep  
Age: 28  
Name: Deep  
Age: 28
```

Topic-8: Dynamic method dispatch

Dynamic method dispatch is a mechanism in Java that allows method calls to be resolved at runtime based on the actual object type, rather than the reference type. This is a key aspect of Java's polymorphism, allowing for more flexible and reusable code.

How Dynamic Method Dispatch Works

When you call an overridden method on a superclass reference that actually points to a subclass object, Java determines which method to execute at runtime based on the object's actual type, not the reference type.

Example of Dynamic Method Dispatch

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {
```

```
Animal myAnimal; // Reference of type Animal

// Upcasting to Animal type
myAnimal = new Dog();
myAnimal.sound(); // Output: Dog barks

myAnimal = new Cat();
myAnimal.sound(); // Output: Cat meows
}
```

Output

Dog barks

Cat meows

Explanation

1. Superclass and Subclasses:

- We define a superclass `Animal` with a method `sound()`.
- Two subclasses, `Dog` and `Cat`, override the `sound()` method with their own implementations.

2. Dynamic Method Dispatch:

- In the `Main` class, we create a reference of type `Animal` but instantiate it as `Dog` and then as `Cat`.
- When `myAnimal.sound()` is called, Java determines at runtime which `sound()` method to execute based on the actual object type (`Dog` or `Cat`), not the reference type (`Animal`).

Topic-9: Abstract class in Java

abstract method:

A method that is declared as abstract and does not have implementation is known as abstract method.

Example

```
abstract void printStatus(); //no body and abstract
```

abstract class:

A class that is declared with abstract keyword is known as abstract class in java. It can have abstract and non-abstract methods (method with body).

Points to Remember

- An abstract class must be declared with an abstract keyword.
- It can have abstract and non-abstract methods.
- It cannot be instantiated.
- It can have constructors and static methods also.
- It can have final methods which will force the subclass not to change the body of the method.

Example:

```
abstract class Shape{
    abstract void draw();
}
class Rectangle extends Shape{
    void draw() {
        System.out.println("drawing  rectangle");
    }
}
class Circle extends Shape{
    void draw() {
        System.out.println("drawing  circle");
    }
}
class TestAbstraction{
    public static void main(String args[]){
        Shape s=new Rectangle();
        s.draw();
        s=new Circle();
        s.draw();
    }
}
```

Output:

```
drawing rectangle
drawing circle
```